# Memory, Addressing and Pointers

*Introduction to ARM Cortex-M Microprocessors* — Chapter 3

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# Why This Chapter Matters

A microcontroller has no instruction that means “turn on the LED”. It only reads and writes numbered locations. So hardware control becomes one question: *which number do I write, and to which address?* This chapter links the C you know to the addresses the processor uses. It is the shortest path from `int x;` to `GPIOA->ODR`.

**After this chapter, you can:**

1. Explain an address, and how it differs from its contents.
2. Use `&` and `*`.
3. Give the size of a byte, halfword and word; compute an array element’s address.
4. Choose the correct fixed-width type.
5. Explain what `const` and `volatile` do to generated code.
6. Write a pointer that reads and writes a hardware register.

© Muním Zabidi

By the end of this chapter, you will understand exactly how these two lines of C turn on a physical pin on your Nucleo board:

```
volatile uint32_t *gpioa_odr = (volatile uint32_t *)0x40020014;
*gpioa_odr |= (1U << 5);     // PA5 rises to 3.3 V -- the user LED
```

Three things in that code need an answer, and each becomes a section of this chapter:

| Question raised | Where this chapter answers it |
|---|---|
| What is `0x40020014`? | Memory-mapped I/O & the 4 GB memory map |
| What does the `*` mean? | Pointers, `&` and `*` |
| Why `volatile`? | When memory changes without C changing it |

Every abstract concept that follows — addresses, pointer arithmetic, fixed-width types, `volatile` — is a necessary tool to make these two lines light the LED.

© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# Memory Is a Numbered Array of Bytes

Memory is organized as a sequence of one-byte locations. Each byte has a unique address, which acts as its location number.

A 32-bit address can identify

$$2^{32} = 4{,}294{,}967{,}296$$

different byte locations.

| | |
|---|---|
| **Address** | *where* a byte is located |
| **Contents** | *what* is stored at that location |

**Keep these two apart**

A house number and the family living in it are different things. An address identifies a location; it is not the value stored there.

© Munim Zabidi

# Storing a Word

A 32-bit word is stored in memory starting at 0x2000 looks like this:

| *Address* | *Contents* |
|---|---|
| 0x2000 | 0xDE |
| 0x2001 | 0xC0 |
| 0x2002 | 0xAD |
| 0x2003 | 0xDE |

Notice that the addresses increase by one because memory is byte-addressable: each address identifies one byte.

A larger value occupies several consecutive byte locations. For example, a 32-bit word occupies four bytes.

So, an address identifies the *first byte* of a multi-byte object. The remaining bytes follow at the next addresses.

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. **A Variable Has an Address**
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# A Variable Has an Address

Suppose `uint32_t x = 42;` and the compiler puts `x` at `0x20000100`. We can ask two different questions:

| | |
|---|---|
| *Where does x live?* | `0x20000100` — the **address** |
| *What value does x contain?* | `42` — the **value** |

In C, `x` names the value, while `&x` produces its address.

## You already know this from digital electronics

An address is the input to a decoder. With memory-mapped I/O, the decoder can select peripheral registers as well as RAM locations.

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. **A Pointer Holds an Address**
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

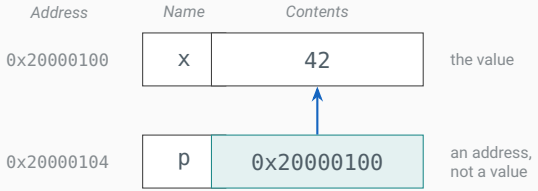
# A Pointer Holds an Address A variable whose value is an address

We just saw that &x gives the address where x lives. C needs somewhere to keep that address. A **pointer** is a variable built for exactly this job.

- &x — address-of: the address at which x is stored
- *p — dereference: the value stored at the address held in p

```
uint32_t  x = 42;
uint32_t *p = &x;    // p holds x's
                     // ADDRESS
uint32_t v = *p;     // v becomes 42
*p = 99;             // x becomes 99
```

These two undo each other: *(&x) is just x.



# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. **Arrays Are Address Arithmetic**
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# Arrays Are Address Arithmetic




An array is a block of elements, placed one after another, all the same size. So its addresses follow a formula.

address of element $i$ =
base + ($i$ × element size)

For `readings[2]`: `0x20000100` $+(2 \times 4) =$ `0x20000108`.

The processor can perform this scaling as part of the memory access instruction.

# Pointer Arithmetic

A pointer already holds an address. Adding an integer to it moves that address by whole elements, not by bytes — because the pointer's type tells C how large one element is.

```
uint32_t *p = &readings[0];
              // p = 0x20000100
p + 1;        // 0x20000104  (+4)
p + 2;        // 0x20000108

uint8_t *q = (uint8_t *)p;
q + 1;        // 0x20000101  (+1)
```

The compiler uses the array formula. It gets the element size from the pointer's own type.

$$p[i] \equiv *(p + i)$$

Array indexing *is* pointer arithmetic.

## Where you will meet this

CMSIS declares the GPIO alternate-function registers as `AFR[2]`. These are two fixed hardware registers. But because indexing is pointer arithmetic, `GPIOA->AFR[1]` computes the right address by itself.

# Three Sentences to Remember

So far, three ideas are enough to explain how C reaches an address of its choosing. Learn these; everything else in this chapter builds on them:

**An array is a memory layout** — consecutive, equally sized objects

**A pointer is an address** — a 32-bit number naming one location

**Indexing is address arithmetic** — base + ($i$ × size)

There is no fourth idea hiding here. A `struct`, seen later, is just a layout whose elements have different sizes. A peripheral register block, later still, is just a struct whose base address is fixed by the chip, not chosen by the compiler. Next: where do these addresses actually lead?

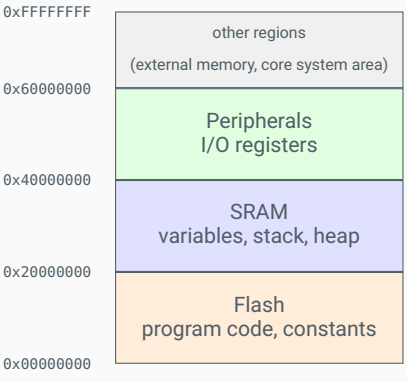
© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. **Where Those Addresses Lead**
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# Where Those Addresses Lead

A Cortex-M microcontroller splits its 4 GB address space into a few large regions:




**Why three main kinds of block?**

- **Flash** is non-volatile: it keeps the program after power is removed, so the chip has something to run at every reset. It reads fast but writes slowly, which fits code that rarely changes.
- **RAM** is volatile but fast to read *and* write: it holds the stack, heap, and variables that change constantly while the program runs. Contents are lost when power is removed.
- **Peripherals** aren't storage at all — each address is wired directly to a hardware register (GPIO, timer, UART, . . . ). Writing there doesn't save a value; it changes the hardware.

*You will spend 99% of your time in the three colored bands above. The grey "other" regions are reserved, bit-band aliases, or external bus space — advanced topics you can safely ignore.*

A pointer, as we just saw, is only a number. So what happens when that number points into the peripheral region?

© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. **Memory-Mapped I/O: The Central Idea**
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# The Central Idea

A peripheral register is a hardware circuit that has been given an address. Reading it returns the circuit's state. Writing it changes the circuit's behaviour.

We already know a pointer is just a number that identifies a location. C can reach any address through pointers. This means C can control hardware directly — no special language feature, no library, no operating system is needed.

```
// name the address as a pointer to a volatile 32-bit register
volatile uint32_t *gpioa_odr = (volatile uint32_t *)0x40020014;

*gpioa_odr |= (1U << 5);    // set bit 5, leave the others alone
```

That is the *whole* mechanism. The cast says: "treat this number as the address of a volatile word." The dereference causes a load and a store. The GPIO hardware latches bit 5. Pin PA5 rises to 3.3 V.

(Why `volatile`? We come back to that shortly.)

# Because All Three Share One Address Space

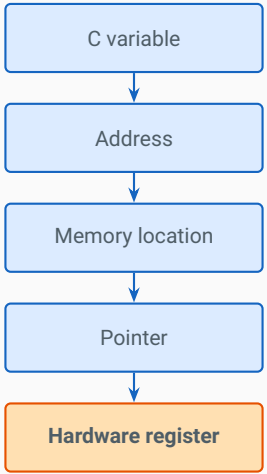
The CPU needs no special instructions for hardware: LDR/STR work everywhere, and the address decoder alone decides whether an access lands in Flash, RAM, or a peripheral register. This is memory-mapped I/O, and it is the reason a plain C pointer is enough to control real hardware.

## Still to answer

How big is one access? And can hardware change a register *without* our code running? The next two sections answer each in turn.

# The Conceptual Spine

This is the conceptual spine of the whole book: how an ordinary C variable becomes a way to control real hardware.




© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. **Choosing Fixed-Width Types**
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. Summary

# Fixed-Width Types

In desktop C, `int` is a reasonable default choice. In embedded C it is a hazard, because the C standard does not fix its width — and every access we just wrote depends on knowing the exact width.

| Type | Width | Sign | Range |
|---|---|---|---|
| `uint8_t` | 8-bit | Unsigned | 0 to 255 |
| `int8_t` | 8-bit | Signed | −128 to 127 |
| `uint16_t` | 16-bit | Unsigned | 0 to 65,535 |
| `uint32_t` | 32-bit | Unsigned | 0 to 4,294,967,295 |
| `int32_t` | 32-bit | Signed | ±2,147,483,647 |

## Two rules

Match the hardware: a 32-bit register is always `uint32_t`. A 12-bit ADC result fits in `uint16_t`.

Then save space, but watch for overflow. If two `uint8_t` values add up to more than 255, the result gets cut off. Add them into a wider type instead.

© Muním Zabidi

# Data Sizes and Alignment

A pointer's type does not only set the step size in pointer arithmetic. It also sets how many bytes one access reads or writes, and where that access is allowed to start.

| Name | Bits | Bytes | C type |
|---|---|---|---|
| Byte | 8 | 1 | `uint8_t` |
| Halfword | 16 | 2 | `uint16_t` |
| Word | 32 | 4 | `uint32_t` |

A word starting at `0x20000000` occupies `0x20000000`–`0x20000003`. The next word starts at `0x20000004`.

## Alignment

- Word access should be at a multiple of 4
- Halfword access at an even address
- Aligned accesses are generally more efficient. Misaligned ones are slower, or cause a fault

## Why every STM32 register offset advances by 4

Each peripheral register is one word. When a datasheet shows offsets `0x00`, `0x04`, `0x08`, this is why: each one sits 4 bytes further than the last.

© Muním Zabidi

# Table of Contents


1 Why This Matters
2 Memory as a Numbered Array of Bytes
3 A Variable Has an Address
4 A Pointer Holds an Address
5 Arrays Are Address Arithmetic
6 Where Those Addresses Lead
7 Memory-Mapped I/O: The Central Idea
8 Choosing Fixed-Width Types
9 **When Memory Can Change Without C Changing It**
10 const: A Brief Aside
11 What a Pointer Becomes
12 Where Variables Live
13 Reading Memory in a Debugger
14 Summary

# When Memory Can Change Without C Changing It

Every C program relies on one basic assumption, so basic that we rarely notice it: *a variable's value changes only when your code changes it.*

1. **An ordinary variable** always obeys this rule. Nothing else can touch it between two reads.
2. **The compiler relies on this too**: it may read a value once, store it in a register, and reuse that register on every later access.
3. **A hardware register breaks this assumption**. It is memory-mapped, so C reaches it through an ordinary pointer — but it changes because a conversion finished, or a timer overflowed, not because of a C assignment.
4. **A variable shared with an ISR** has the same problem: to the main program, a flag the ISR sets seems to change with no matching write.
5. **`volatile` is the fix**: it tells the compiler to re-read this address every time, and to perform every write, in order, with none skipped.

## The statement to remember

`volatile` tells the compiler a value may change outside the current sequence of C statements. Hardware registers are just the most common reason this happens in embedded code.

© Muním Zabidi

```c
volatile uint32_t *status = (volatile uint32_t *)0x40011000;
while ((*status & 1) == 0) {
    // Without volatile the compiler reads the address once, caches it,
    // and this loop never terminates no matter what the hardware does.
}
```

## The rule

*Every pointer to a hardware register, and every variable shared with an interrupt handler, must be `volatile`.*

Chapter 9 uses this same keyword for a different situation: an ISR talks to the main loop using nothing more than a flag.

© Munim Zabidi

# Spot the Bug Why might this loop never exit?

The hardware sets bit 0 of the status register as soon as a sample is ready. Yet the program below sometimes spins forever, even on hardware that is clearly doing its job.

```
uint32_t *status = (uint32_t *)0x40011000;    // note: NOT volatile
while ((*status & 1) == 0) {
    /* wait for sample */
}
```

## Your prediction

- What does an optimizing compiler assume about `*status` across loop iterations?
- How many times does it actually read the address?
- What would happen if the compiler kept the value in a register?

**The answer** The compiler sees that nothing inside the loop writes to `*status`, so by assumption 1 of the previous slide, one read is enough: it caches the first value in a register and never touches the address again. The hardware changes memory, but the register never changes — the loop condition is false forever.

**Fix:** declare `status` as `volatile uint32_t *`.

© Muním Zabidi

# Table of Contents


1 Why This Matters
2 Memory as a Numbered Array of Bytes
3 A Variable Has an Address
4 A Pointer Holds an Address
5 Arrays Are Address Arithmetic
6 Where Those Addresses Lead
7 Memory-Mapped I/O: The Central Idea
8 Choosing Fixed-Width Types
9 When Memory Can Change Without C Changing It
10 **const: A Brief Aside**
11 What a Pointer Becomes
12 Where Variables Live
13 Reading Memory in a Debugger
14 Summary

# A Brief Aside: `const`

`const` is a promise: the program will not change this object. The practical benefit is placement: a `const` array can live in Flash instead of SRAM, so it costs no scarce RAM.

```
const uint8_t sine_table[256] = {
    128, 131, 134, /* ... */
};  // lives in Flash
```

That is all there is to `const`. It says nothing about *when* a value can change, which is exactly the question `volatile` answers.

# Table of Contents


1 Why This Matters
2 Memory as a Numbered Array of Bytes
3 A Variable Has an Address
4 A Pointer Holds an Address
5 Arrays Are Address Arithmetic
6 Where Those Addresses Lead
7 Memory-Mapped I/O: The Central Idea
8 Choosing Fixed-Width Types
9 When Memory Can Change Without C Changing It
10 const: A Brief Aside
11 **What a Pointer Becomes**
12 Where Variables Live
13 Reading Memory in a Debugger
14 Summary

# What a Pointer Becomes

Every pointer operation compiles into one of exactly two instructions: a **load** or a **store**.

| C | Instruction | What happens |
|---|---|---|
| `v = *p;` | `LDR R0, [R1]` | Read the word at the address in R1 |
| `*p = v;` | `STR R0, [R1]` | Write R0 to the address in R1 |
| `v = p[2];` | `LDR R0, [R1, #8]` | Read the word 8 bytes past R1 |
| `c = *bytePtr;` | `LDRB R0, [R1]` | Read one *byte*, zero-filled |

## A dereference is a direction

On the right of an assignment, it is a **load**. On the left, it is a **store**. The pointer itself is just a 32-bit number in a register, supplying the address.

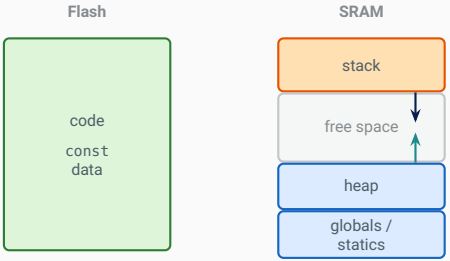
This is the mechanical end of the story: a pointer dereference is not an abstraction, it *is* one of these two instructions.

# Table of Contents


1 Why This Matters
2 Memory as a Numbered Array of Bytes
3 A Variable Has an Address
4 A Pointer Holds an Address
5 Arrays Are Address Arithmetic
6 Where Those Addresses Lead
7 Memory-Mapped I/O: The Central Idea
8 Choosing Fixed-Width Types
9 When Memory Can Change Without C Changing It
10 const: A Brief Aside
11 What a Pointer Becomes
12 **Where Variables Live**
13 Reading Memory in a Debugger
14 Summary

# Where Variables Live

A pointer is an address — but who decides *which* address a variable gets? **Must understand:** the stack, global/static storage, Flash, SRAM. **Should understand:** the heap.




## The stack

- Locals, arguments beyond the first four, saved registers, return addresses
- Managed automatically by SP (R13)
- Grows downward from the top of SRAM
- Chapter 7 builds an entire chapter on it

## The heap

- Gives memory through `malloc()`, freed by `free()`
- Usually avoided in bare-metal work: it can fail at runtime when memory fragments, and its timing is not predictable
- Almost no example in this book calls `malloc`

## Stack overflow — a classic embedded failure

The downward-growing stack can collide with the upward-growing global data. This silently corrupts variables. The usual causes are deep recursion and large local arrays.

© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. **Reading Memory in a Debugger**
14. Summary

Consider:

```
uint32_t readings[4] = { 10, 20, 30, 40 };     // suppose at 0x20000100
```

A memory window shows the bytes at those addresses directly:

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|---|---|---|---|---|---|---|---|---|
| 0x20000100 | 0A | 00 | 00 | 00 | 14 | 00 | 00 | 00 |
| 0x20000108 | 1E | 00 | 00 | 00 | 28 | 00 | 00 | 00 |

Row address + column offset gives the byte address. Column `04` does not mean address `0x04` — it is an offset of 4 bytes from the row's address. This is the same arithmetic as `readings[1]` at `0x20000100 + 1×4`.

© Muním Zabidi

# Bytes Are Not the Same as Values

A debugger shows one byte per cell; a C variable may be larger than one byte. These are two views of the same object.

`uint32_t x = 42;` where $42 = 0x0000002A$. On the Cortex-M4 (little-endian):

| Address | +0 | +1 | +2 | +3 |
|---|---|---|---|---|
| 0x20000100 | 2A | 00 | 00 | 00 |

### Three questions to ask of a dump

1. What address does this byte occupy?
2. How many bytes belong to the object?
3. How does the object's type interpret those bytes?

The same bytes mean different values when interpreted with different types.

## A debugging habit

When a variable has an unexpected value, inspect the bytes in memory first, not the C expression that produced it. Vague "why is it wrong?" becomes concrete questions about addresses, bytes, and type.

Given the declaration below, placed at `0x20000100`:

```
uint32_t readings[4] = { 10, 20, 30, 40 };
```

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|---|---|---|---|---|---|---|---|---|
| 0x20000100 | ? | ? | ? | ? | ? | ? | ? | ? |
| 0x20000108 | ? | ? | ? | ? | ? | ? | ? | ? |

**Fill in the sixteen cells.**

1. At what address does `readings[2]` begin?
2. Which four cells does its value occupy?
3. Which byte, in little-endian order, holds the `0x1E`?

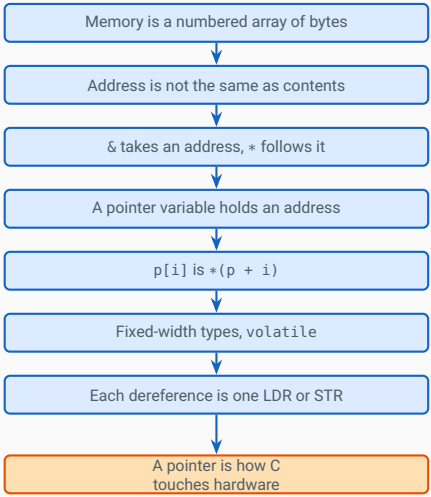
**Answers**: `readings[2]` is at `0x20000100 + 2×4 = 0x20000108`, occupying cells `+00..+03` of the second row. The byte `1E` lives at `0x20000108` itself — least-significant byte first.

© Muním Zabidi

# Table of Contents


1. Why This Matters
2. Memory as a Numbered Array of Bytes
3. A Variable Has an Address
4. A Pointer Holds an Address
5. Arrays Are Address Arithmetic
6. Where Those Addresses Lead
7. Memory-Mapped I/O: The Central Idea
8. Choosing Fixed-Width Types
9. When Memory Can Change Without C Changing It
10. const: A Brief Aside
11. What a Pointer Becomes
12. Where Variables Live
13. Reading Memory in a Debugger
14. **Summary**

# Summary




**Terminology covered**

- Address, byte, halfword, word
- Pointer, &, *
- Fixed-width types
- `const`, `volatile`

**Next: Chapter 4**

*Embedded C and CMSIS*

We use this mechanism to turn on a real LED.